

A bit of theory as far as relevant to the implementation of `class LagrangeMesh`

The necessary theory - for non-reduced axes - is found in [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation. The corresponding section in [Baye, Phys. Rep. 565, 1 \(2015\)](#) is 3.7.1. Lagrange–Fourier mesh.

Note

The figures in this document are created by the tests. If the document complains `blablablah.png could not be found.`, please run the tests (`pytest tests` from folder `MOCCaPy`)

Non-reduced axes

(As from [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation)

Grid points

Using M evenly space grid point at distance Δ , the boundaries of the box are $[-\frac{M}{2}\Delta, \frac{M}{2}\Delta]$. The width of the box is $L = M\Delta$. We require $M = 2N$ to be even such that there are $N = \frac{M}{2}$ grid points on each side of the origin and the origin is avoided as a grid point.

For non-reduced axes (where the shift parameter σ is required be zero) the grid points are:
[eq 1]

$$\pm \frac{1}{2}\Delta, \pm \frac{3}{2}\Delta, \dots, \pm \frac{2N-3}{2}\Delta, \pm \frac{2N-1}{2}\Delta$$

or

[eq 2]

$$x_{\pm i} = \pm \left(\frac{1}{2} + i\right)\Delta = \pm \left(\frac{2i+1}{2}\right)\Delta$$

$$i = 0..N-1$$

(the notation $x_{\pm i}$ is not entirely accurate as it must distinguish between $+0$ and -0 .)

In increasing order:

[eq 3]

$$-\frac{2N-1}{2}\Delta, -\frac{2N-3}{2}\Delta, \dots, -\frac{3}{2}\Delta, -\frac{1}{2}\Delta, \frac{1}{2}\Delta, \frac{3}{2}\Delta, \dots, \frac{2N-3}{2}\Delta, \frac{2N-1}{2}\Delta$$

or

[eq 4]

$$x_i = -\frac{2N-1-2i}{2}\Delta = \left(\frac{1}{2} + i - N\right)\Delta$$
$$i = 0..(2N-1)$$

or

[eq 4.1]

$$x_i = -\frac{2N-1-2i}{2}dx = \left(\frac{1}{2} + i - N\right)dx$$
$$i = -(N-1)..(N-1)$$

Basis functions

The basis functions are plane waves:

[eq 5]

$$\phi_k(x) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L} kx\right)$$

where we choose j for the imaginary unit (rather than j , as j and j are a bit better distinguishable than j and i), and

$$k = \pm\frac{1}{2}, \pm\frac{3}{2}, \dots, \pm\frac{2N-3}{2}, \pm\frac{2N-1}{2}$$

or, since $k\Delta$ is a grid point, say, the i -th, x_i :

[eq 5]

$$\phi_{x_i}(x) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L} \frac{x_i}{\Delta} x\right) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L\Delta} x_i x\right)$$

Note

Since $\exp(jz) = \cos z + j \sin z$ the real component of the 1D basis function is symmetric and the imaginary component is skew-symmetric. Consequently, we can use them for testing interpolation with Lagrange functions (see below) also on reduced grids.

$$\text{Re}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \cos\left(\frac{2\pi}{L\Delta} x_i x\right)$$

$$\text{Im}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \sin\left(\frac{2\pi}{L\Delta} x_i x\right)$$

First order derivatives are:

$$\frac{d}{dx} \text{Re}(\phi_{x_i}(x)) = -\frac{1}{\sqrt{L}} \frac{2\pi}{L\Delta} \sin\left(\frac{2\pi}{L\Delta} x_i x\right)$$

$$\text{Im}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \frac{2\pi}{L\Delta} \cos\left(\frac{2\pi}{L\Delta} x_i x\right)$$

Interpolation

The Lagrange interpolation functions are:

[eq 6]

$$f_i(x) = \frac{1}{2N} \frac{\sin\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)}$$

By construction, the Lagrange interpolation functions $f_i(x)$ have the property of being equal to 1 at the i -th mesh point, x_i , and 0 at all others:

[eq 7]

$$f_i(x_j) = \delta_{ij}$$

Note that when evaluating $f_i(x_i)$, both the numerator and the denominator are 0, and Python yields a `nan`. We need to invoke l'Hôpital's rule to obtain the result.

[eq 7.1]

$$\begin{aligned} & \lim_{x \rightarrow x_i} \frac{1}{2N} \frac{\sin\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\sin'\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin'\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\frac{\pi}{\Delta} \cos\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\frac{\pi}{2N\Delta} \cos\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{2N}{2N} \lim_{x \rightarrow x_i} \frac{\cos(0)}{\cos(0)} = 1 \end{aligned}$$

In order to avoid testing for $\frac{0}{0}$ it suffices to add $\varepsilon \approx 1e - 9$ to x_i to avoid the `nan` and yield something close to 1.

The arguments of the two sine functions in eq 6 are the same, apart from a factor $1/2N$:

[eq 8]

$$\begin{aligned} f_i(x) &= \frac{1}{2N} \frac{\sin(A_i(x))}{\sin\left(\frac{A_i(x)}{2N}\right)} \\ A_i(x) &= \frac{\pi}{\Delta}(x - x_i) \end{aligned}$$

This can be exploited for efficiency in a computation.

An arbitrary function $h(x)$ taking the values $h_i = h(x_i)$ on the grid points can be interpolated on an arbitrary point $x \in [-N\Delta, N\Delta]$ as:

[eq 9]

$$h(x) = \sum_{i=0}^{2N-1} h(x_i) f_i(x)$$

This is essentially a dot product $\mathbf{f} \cdot \mathbf{h}$.

Derivatives

The 1st and 2nd order derivatives of the Lagrange interpolation functions are:

[eq 10.1]

$$D_{ji}^{(1)} = \left. \frac{df_i(x)}{dx} \right|_{x=x_j} = \begin{cases} (-1)^{i-j} \frac{\pi}{2N\Delta} \frac{1}{\sin(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ 0, & \text{for } i = j. \end{cases}$$

[eq 10.2]

$$D_{ji}^{(2)} = \left. \frac{d^2 f_i(x)}{dx^2} \right|_{x=x_j} = \begin{cases} (-1)^{i-j+1} 2 \left(\frac{\pi}{2N\Delta} \right)^2 \frac{\cos(\pi(i-j)/2N)}{\sin^2(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ -\frac{\pi^2}{3\Delta^2} \left(1 - \frac{1}{(2N)^2} \right), & \text{for } i = j. \end{cases}$$

⚠ Warning

I suspect that there is a subtle problem with these formulas. When coding them the unit test failed giving the right values but the opposite sign. While attempting to derive the formulas i discovered that using the definition in eq 6, taking derivatives and evaluating in x_j leads to $(x_j - x_i)$ in the arguments of the sine functions, which is equal to $(j - i)\Delta$, and NOT $(i - j)$ as written in the formulas. The derivation proceeds by considering separate cases for odd and even $(j - i)$ and noting that $\cos \pi(2n)$ and $\sin \pi(2n + 1)$ vanish, yielding the sign factor in eq 10.1 and 10.2., $(-1)^{(i-j)} = (-1)^{(j-i)}$, but obviously the change from $(i - j)$ to $(j - i)$ in the sign functions yields a sign flip. (suspicion confirmed)

By applying these to the expansion $\phi(x) = \sum_{i=0}^{2N-1} \phi(x_i) f_i(x)$ we obtain

[eq 11]

$$\frac{d\mathbf{h}}{dx} = \left. \frac{dh(x)}{dx} \right|_{x=x_j} = \sum_{i=0}^{2N-1} \left. \frac{df_i(x)}{dx} \right|_{x=x_j} h(x_i) = \sum_{i=0}^{2N-1} D_{ji}^{(1)} h(x_i) = \mathbf{D}^{(1)} \mathbf{h}$$

So, the column vector $\frac{d\mathbf{h}}{dx}$ of the derivatives of $\mathbf{h}$ at all grid points is found as a matrix product of $\mathbf{D}^{(1)}$ with the column vector of the values of $\mathbf{h}$ at all the gridpoints. As the 2nd derivative forms a matrix as well, we also have

[eq 12]

$$\frac{d^2 \mathbf{h}}{dx^2} = \mathbf{D}^{(2)} \mathbf{h}$$

Note, that single differentiation toggles the symmetry behavior of the function: if $h(x)$ is a

symmetric function, $h'(x)$ is skew-symmetric, and *vice versa*. Consequentially, double differentiation toggles it twice, hence keeps the symmetry behaviour the same.

Reduced axes

Grid points

On reduced axes only the N grid points to the right of the origin are kept:

[eq 13]

$$\frac{1}{2}dx, \frac{3}{2}dx, \dots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or

[eq 14]

$$x_i = (i + \frac{1}{2})dx$$

$$i = 0..N-1$$

(In the case of reduced axis a shift $\sigma < dx$ may be subtracted from each grid point.)

Interpolation

According to (eq 9) an arbitrary function $h(x)$ can be interpolated as:

$$h(x) = \sum_{i=0}^{2N-1} h(x_i) f_i(x)$$

If the x -axis is reduced, i.e. we require $h(x)$ to be symmetric ($h(-x) = h(x)$) or skew-symmetric ($h(-x) = -h(x)$), then the above equation becomes:

$$h(x) = \sum_{i=0}^{N-1} h(x_i) f_i(x) + \sum_{i=0}^{N-1} h(-x_i) f_{-i}(x)$$

where $f_{-i}(x)$ is the Lagrange interpolation function corresponding to the i -th grid point to the left of the origin

[eq 14.1]

$$f_{-i}(x) = \frac{1}{2N} \frac{\sin(\frac{\pi}{\Delta}(x - x_{-i}))}{\sin(\frac{\pi}{\Delta} \frac{x - x_{-i}}{2N})} = \frac{1}{2N} \frac{\sin(\frac{\pi}{\Delta}(x + x_i))}{\sin(\frac{\pi}{\Delta} \frac{x + x_i}{2N})}$$

(where again we must distinguish a bit sloppily between $i = +0$ and $i = -0$ as the first mesh point index right, resp. left of the origin).

This then becomes:

$$h(x) = \sum_{i=0}^{N-1} \begin{cases} h(x_i)(f_i(x) + f_{-i}(x)), & h \text{ is symmetric} \\ h(x_i)(f_i(x) - f_{-i}(x)), & h \text{ is skew-symmetric} \end{cases}$$

or

[eq 15]

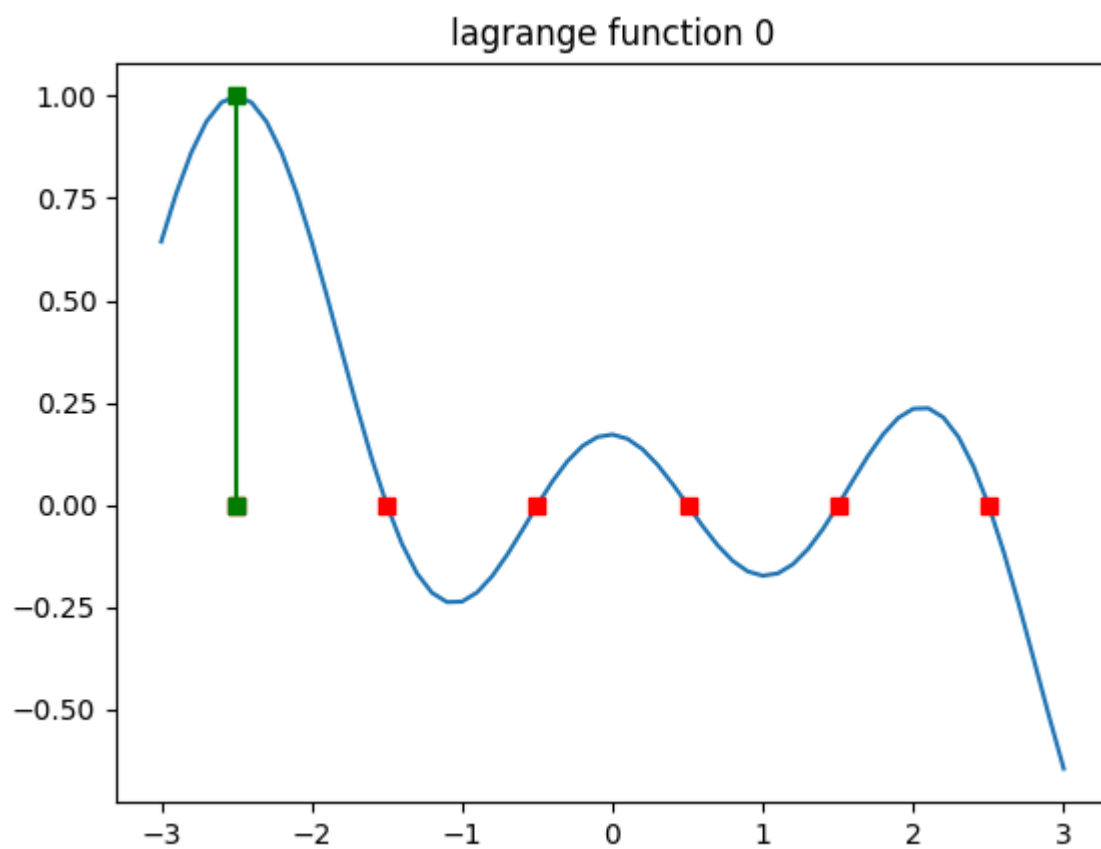
$$h(x) = \sum_{i=0}^{N-1} h(x_i)(f_i(x) \pm f_{-i}(x)) = \mathbf{h} \cdot \mathbf{f}_{\pm}$$

where the $\pm$ is $+$ for the symmetric case and $-$ for the skew-symmetric case, and $\mathbf{f}_{\pm} = \mathbf{f}_+ \pm \mathbf{f}_-$

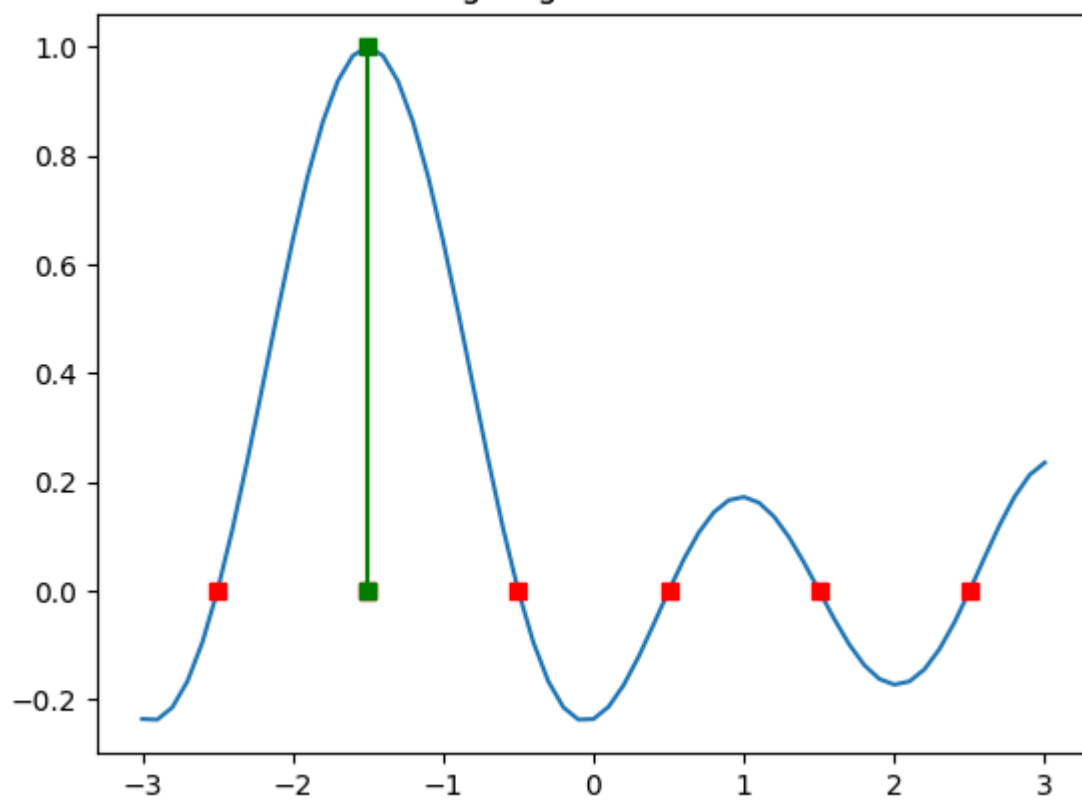
.

General remark on interpolation with Lagrange functions

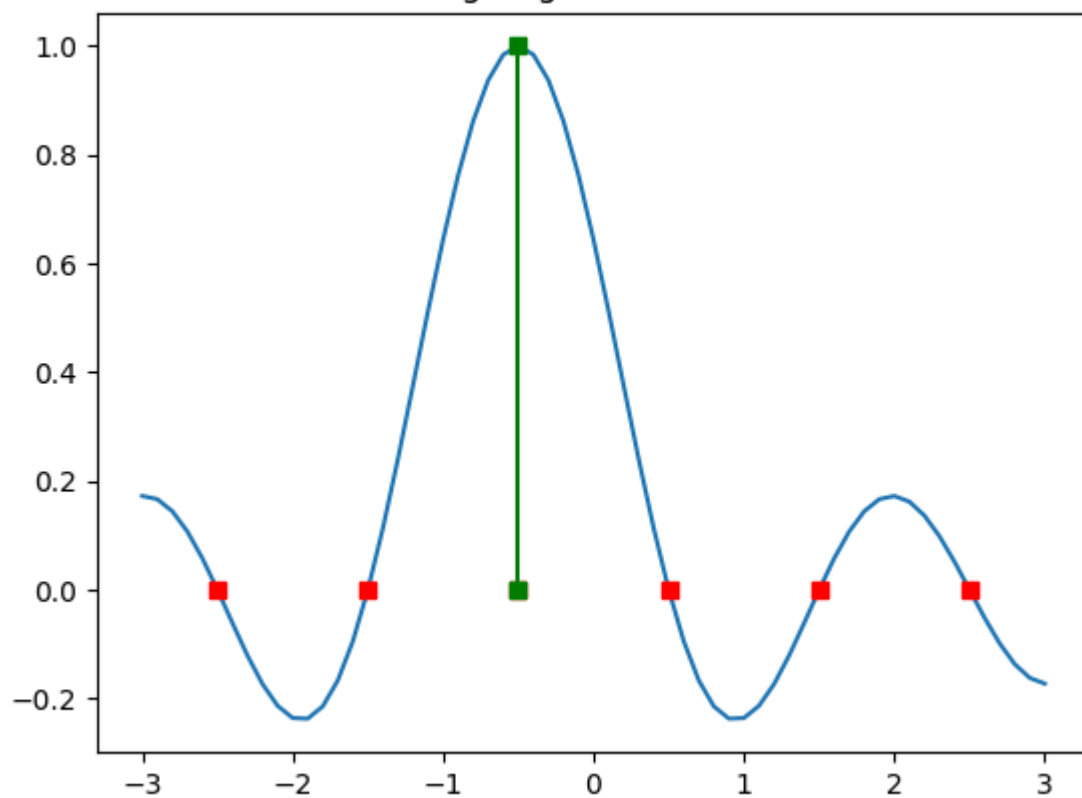
On a 1D grid with 6 grid points at $-1.25, -.75, -.25, .25, .75, 1.25$ on the interval $[-1.5, 1.5]$, these are the 6 Lagrange functions:



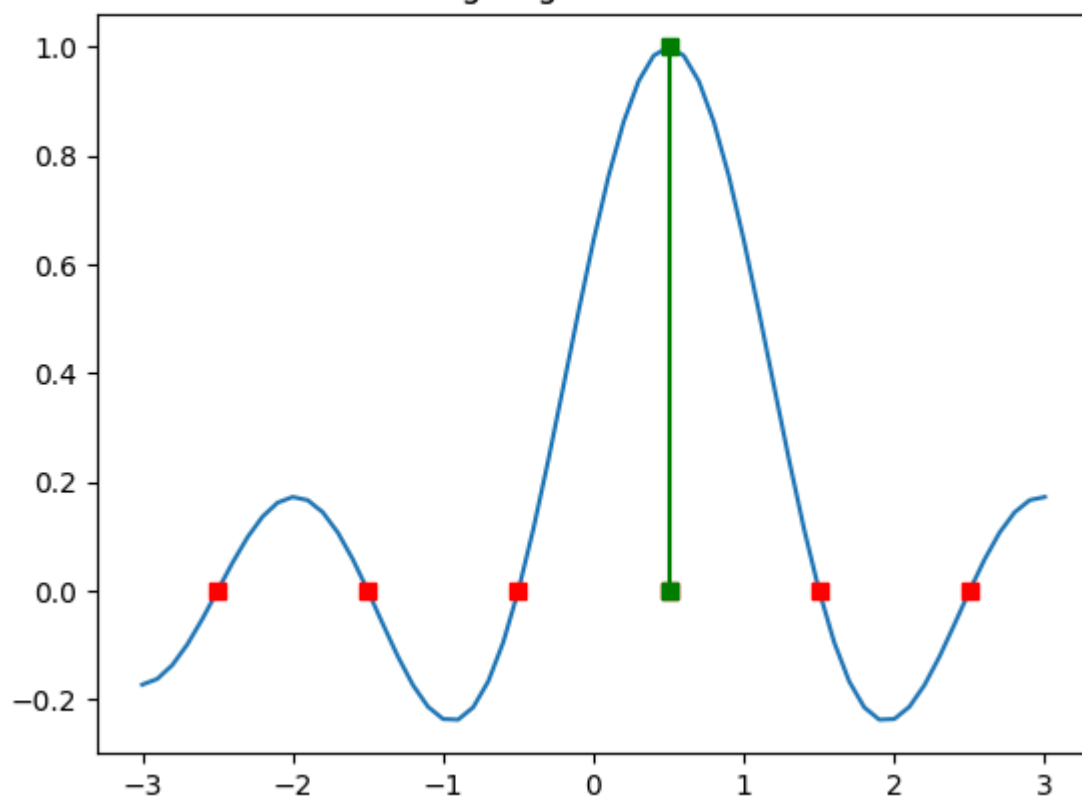
lagrange function 1



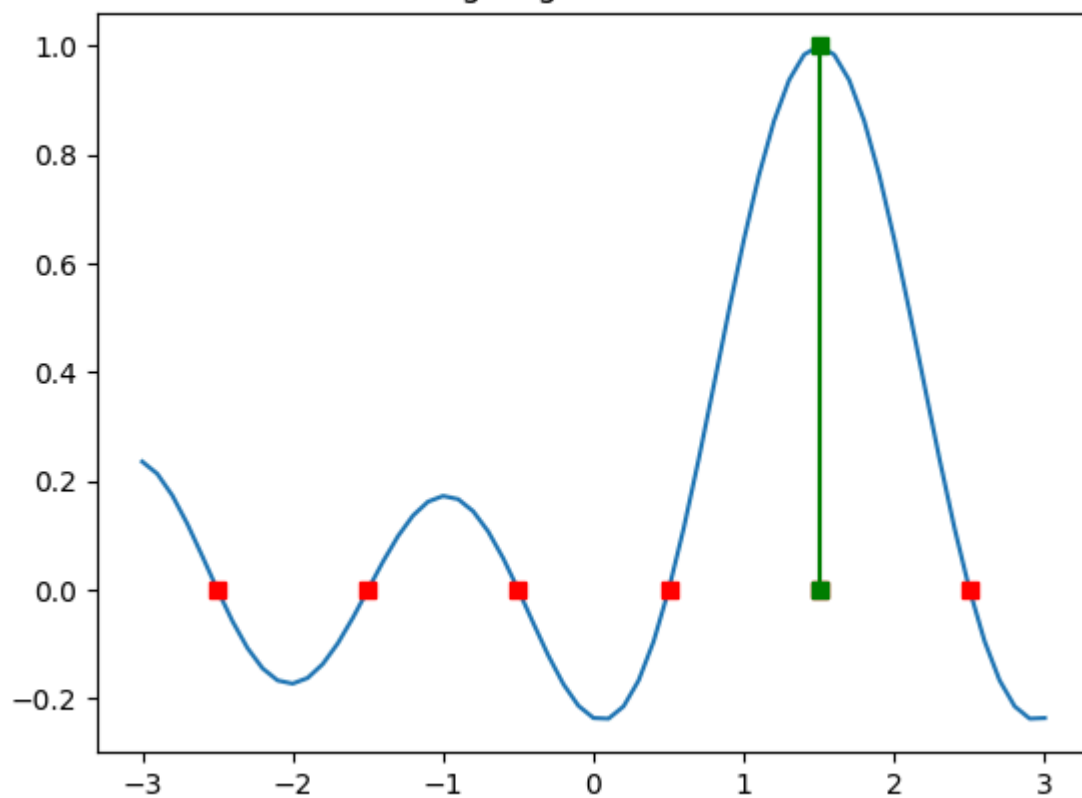
lagrange function 2

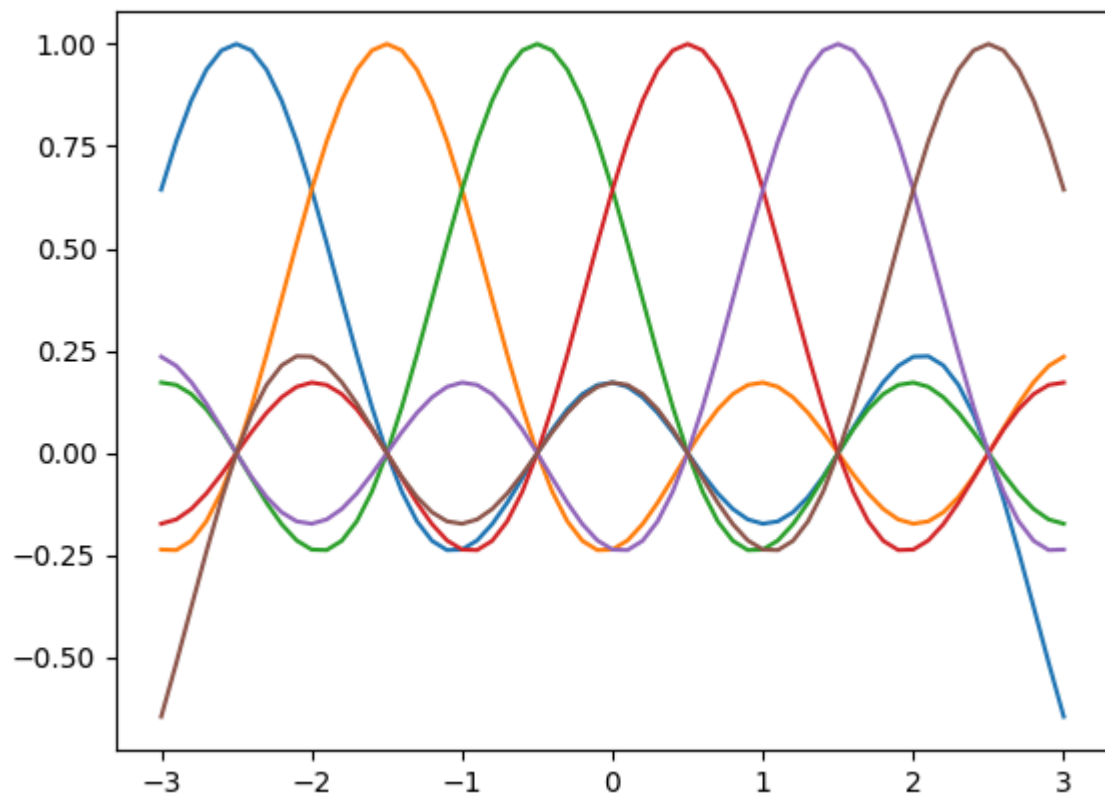
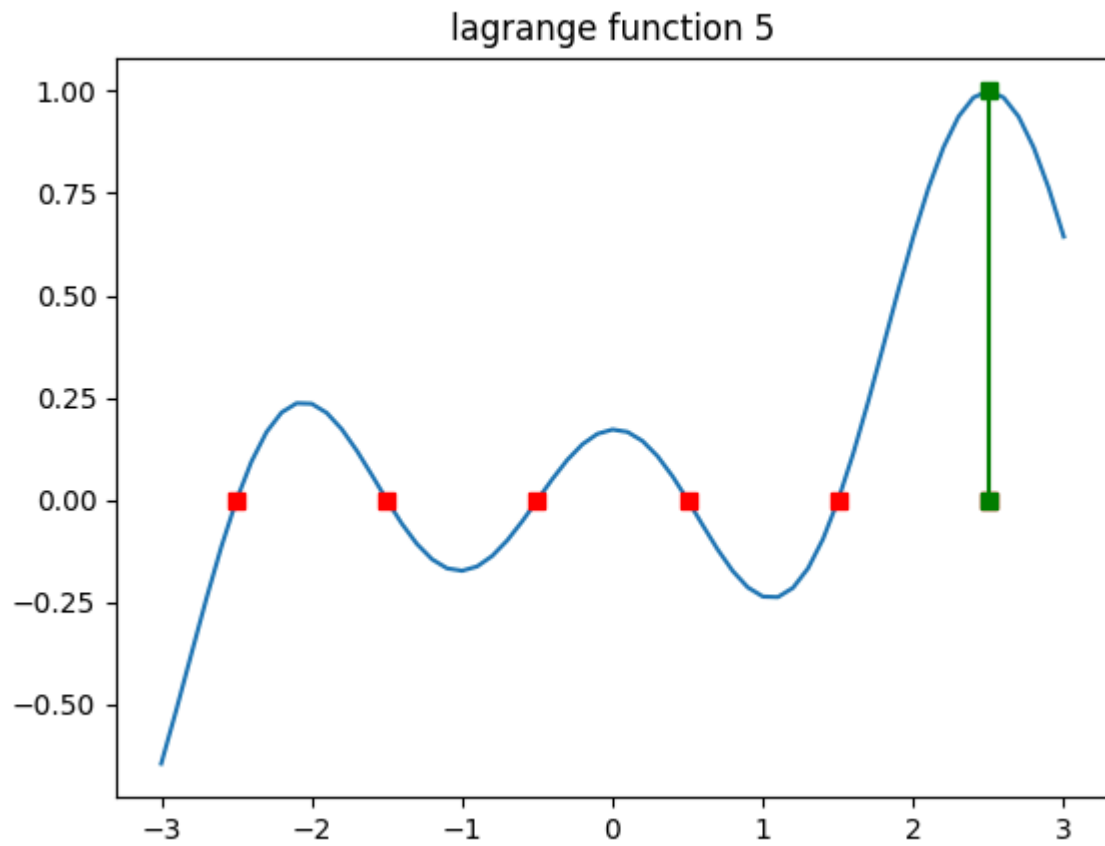


lagrange function 3



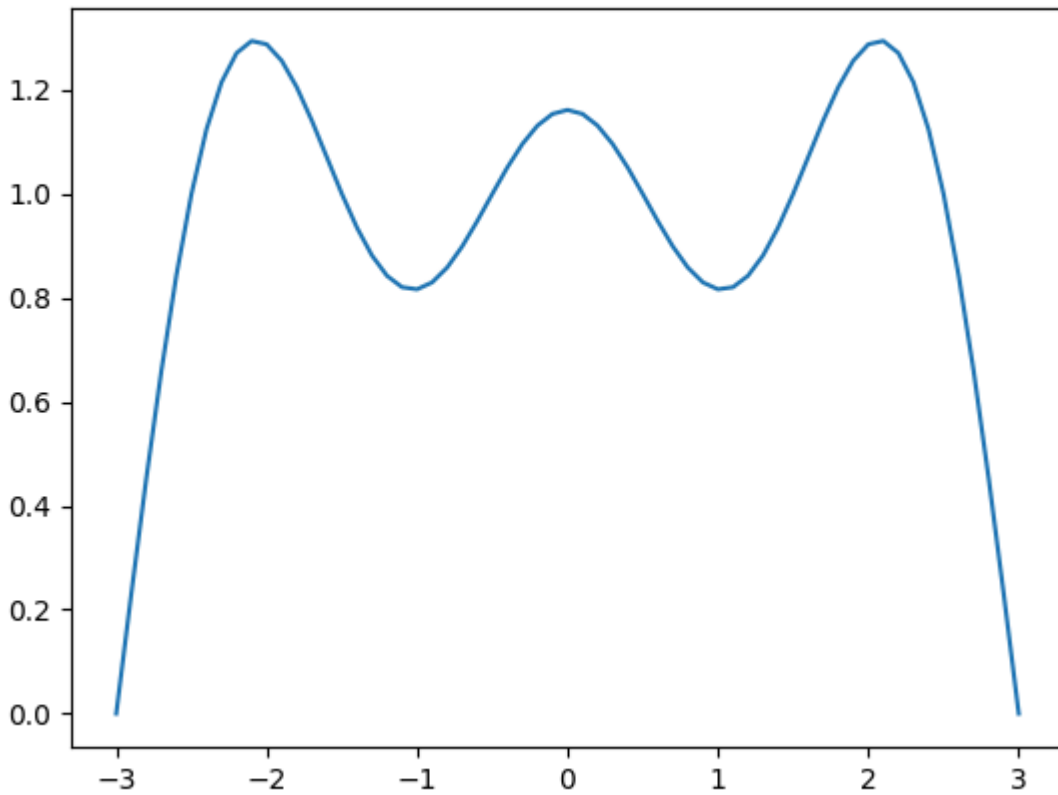
lagrange function 4





Clearly, each one yields 1 at one grid point and 0 at the others. Note also that they are antiperiodic: they reenter the box at the opposite edge, but **with a sign change**. Consequently, a constant function cannot be interpolated because it is periodic. The sum of

the 6 Lagrange functions is shown below. it is definitely not the constant function $f(x) = 1$.



According to the discussion in [github issue 52](#) periodic functions can be interpolated with Lagrange functions provided they vanish at the boundary of the interval.

Derivatives

The formula for the derivative of a function h expanded on a reduced grid is found easily by extending the column vector $\mathbf{h}$ (of length N) on the reduced grid as

[eq 17]

$$\begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix}$$

with $\mathbf{g}$ containing the same elements as $\mathbf{h}$ but in reverse order, as if they were mirrored by a horizontal line between the two. The full vector then corresponds to the $\mathbf{h}_{\text{non-reduced}}$ case.

This allows for the matrix product :

[eq 18]

$$\mathbf{D} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \\ \mathbf{D}^+ \end{bmatrix} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \\ \mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \end{bmatrix}$$

The upper half of $\mathbf{D}$, $\mathbf{D}^-$, produces the derivatives on the negative x -axis, and the lower half of $\mathbf{D}$, $\mathbf{D}^+$, produces the derivatives on the positive x -axis.

Note

It can be demonstrated that $\mathbf{D}^T = -\mathbf{D}$, but apart from that $\mathbf{D}^-$ and $\mathbf{D}^+$ are unrelated. Thus $\mathbf{D}$ cannot be reduced to a rectangular matrix that can be applied to a reduced $\mathbf{h}$. (E.g. the transpose of the lower half of $\mathbf{D}$ cannot recover its upper left quadrant.)

The obvious way to proceed with differentiation for reduced axes is to construct the full $\mathbf{D}$ -matrix and then applying eqs 17 and 18.

Note

According to eqs 10.1 and 10.2 the $\mathbf{D}$ -matrix depends on the mesh only through N (or $M = 2N$) and Δ , not on the coordinates of the individual mesh points. So, its computation need not distinguish between the reduced and non-reduced case.

Because of the symmetry properties of h and differentiation, we only need, in fact

$$\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \mathbf{D}^+ \mathbf{h}^\dagger$$

An interesting question, from the performance point of view, is whether we can compute this result without explicitly constructing

$$\mathbf{h}^\dagger = \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix}$$

which requires copying $\mathbf{h}$ twice.

We have, with $i = 0, \dots, 2N - 1$:

$$\left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i = \sum_{l=0}^{l=2N-1} D_{il}^+ h_l^\dagger$$

$$= \sum_{l=0}^{l=N-1} D_{il}^+ h_l^\dagger + \sum_{l=N}^{l=2N-1} D_{il}^+ h_{-l}^\dagger$$

$$= \sum_{l=0}^{l=N-1} D_{il}^+ h_l^\dagger + (\pm h_{N-1-l})$$

$$+ \sum_{l=N}^{l=2N-1} D_{il}^+ h_{-l}^\dagger$$

$$= \pm \sum_{l=0}^{l=N-1} D_{il}^+ h_{N-1-l}^\dagger$$

$$+ \sum_{l=0}^{l=N-1} D_{i,N+l}^+ h_l^\dagger$$

$$f_{diid} \mathbf{D} \in N \times N \text{ dms } \mathbf{D} = \begin{bmatrix} \mathbf{D}^{\text{ul}} & \mathbf{D}^{\text{ur}} \\ \mathbf{D}^{\text{ll}} & \mathbf{D}^{\text{lr}} \end{bmatrix}$$

$$\left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i = \pm \sum_{l=0}^{l=N-1} D_{il}^{\text{ll}} h_{N-1-l} + \sum_{l=0}^{l=N-1} D_{i,l}^{\text{lr}} h_l$$

now with $i = 0, \dots, N-1$. The second sum can be implemented using `numpy.einsum`, as in the non-reduced case. For the first sum, this is perhaps not possible because the access to is reversed. We can eliminate the reversed access to h , by reversing the access to D_{ij}^{ll} . [eq 19]

$$\begin{aligned} \left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i &= \pm \sum_{l=0}^{l=N-1} D_{il}^{\text{ll}} h_{N-1-l} + \sum_{l=0}^{l=N-1} D_{il}^{\text{lr}} h_l \\ &= \pm \sum_{l=0}^{l=N-1} D_{i,N-1-l}^{\text{ll}} h_l + \sum_{l=0}^{l=N-1} D_{il}^{\text{lr}} h_l \\ &= \sum_{l=0}^{l=N-1} [\pm D_{i,N-1-l}^{\text{ll}} + D_{il}^{\text{lr}}] h_l \\ &= \sum_{l=0}^{l=N-1} [\pm E_{il}^{\text{ll}} + D_{il}^{\text{lr}}] h_l = \sum_{l=0}^{l=N-1} [D_{il}^{\text{lr}} \pm E_{il}^{\text{ll}}] h_l \end{aligned}$$

Thus, by reversing the order of the columns in D_{ij}^{ll} we can avoid explicit construction of $\mathbf{h}^\dagger$, and allow for an `numpy.einsum` implementation. This is a one time cost, when the $\mathbf{D}$ matrices are constructed.

N-dimensional grids

The full Cartesian 3D representation of a function $h(\mathbf{r})$, where $\mathbf{r} = [x \ y \ z]$ (3D case), is then provided (for the 3D case) by [eq 23]

$$h(\mathbf{r}) = \sum_{ijk} h_{ijk} f_i(x) f_j(y) f_k(z)$$

where the number of discretization points does not have to be the same in each direction.

Note that f_i , f_j and f_k are generally different objects, even if accidentally the indices i , j and k are identical, as they pertain, resp., to the x -axis, the y -axis and the z -axis, and the grid spacing is not necessarily the same.

Derivatives

In this case, the derivative matrices $\mathbf{D}^{(1)}$ and $\mathbf{D}^{(2)}$ have to be set up separately for each direction, taking into account whether the axis is reduced or not.

[eq 23.1]

$$\left. \frac{d\Phi}{dx} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ljk}$$

$$\left. \frac{d\Phi}{dy} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ilk}$$

$$\left. \frac{d\Phi}{dz} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ijl}$$

The summation is over the index of Φ_{ijk} that corresponds to the axis wrt which the derivative is taken: the derivative wrt $x/y/z$ sums over the 1st/2nd/3rd index. The same holds for 2nd, 3rd, 4th order derivatives ($\frac{d^n}{d^n}, n = x, y, z, n \geq 0$).

Note

`numpy.einsum` is ideally suited to compute sums like these.

Basis functions

The basis functions are plane wave products of the different axes:

[eq 24]

$$\begin{aligned} \Phi_{kl}(x, y, z) &= \phi_k(x) \phi_l(y) \phi(z) = \frac{1}{\sqrt{L_x}} \frac{1}{\sqrt{L_y}} \frac{1}{\sqrt{L_z}} \exp\left(\frac{2\pi j}{L_x} kx\right) \exp\left(\frac{2\pi j}{L_y} ly\right) \exp\left(\frac{2\pi j}{L_z} z\right) \\ &= \frac{1}{\sqrt{L_x L_y L_z}} \exp\left(2\pi j \left(\frac{kx}{L_x} + \frac{ly}{L_y} + \frac{z}{L_z}\right)\right) \\ k &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_x - 1}{2} \\ l &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_y - 1}{2} \\ &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_z - 1}{2} \\ &= \frac{1}{\sqrt{L_x L_y L_z}} \exp(2\pi j (\mathbf{r} \cdot \mathbf{k})) \\ &= \left[\frac{k}{L_x} \quad \frac{l}{L_y} \quad \frac{1}{L_z} \right] \\ &= [x \quad y \quad z] \end{aligned}$$

Note that ϕ_k , ϕ_l and ϕ are generally different objects, even if accidentally the indices k , l and are identical, as they pertain, resp., to the x -axis, the y -axis and the z -axis.

Alternatively, in the spirit of eq 5.1:

[eq 24.1]

$$= \left[\frac{x_i}{L_x \Delta x} \quad \frac{y_j}{L_y \Delta y} \quad \frac{z_k}{L_z \Delta z} \right]$$

where the x_i , y_j , z_k are grid coordinates in the x , y and z directions.

 Note

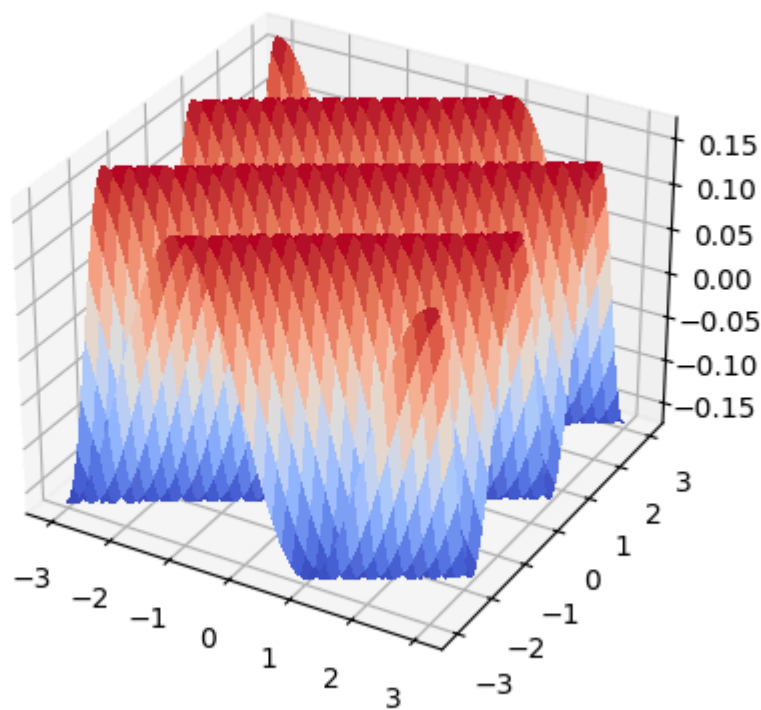
Contrary to the 1D case, it is generally not true for the 2D and 3D cases that the basis functions are symmetric or skew-symmetric.

E.g. changing the sign of the x -coordinate in $\exp(2\pi j(\cdot \mathbf{r})) = \exp(2\pi j(\cdot [-x, y, z]))$ yields

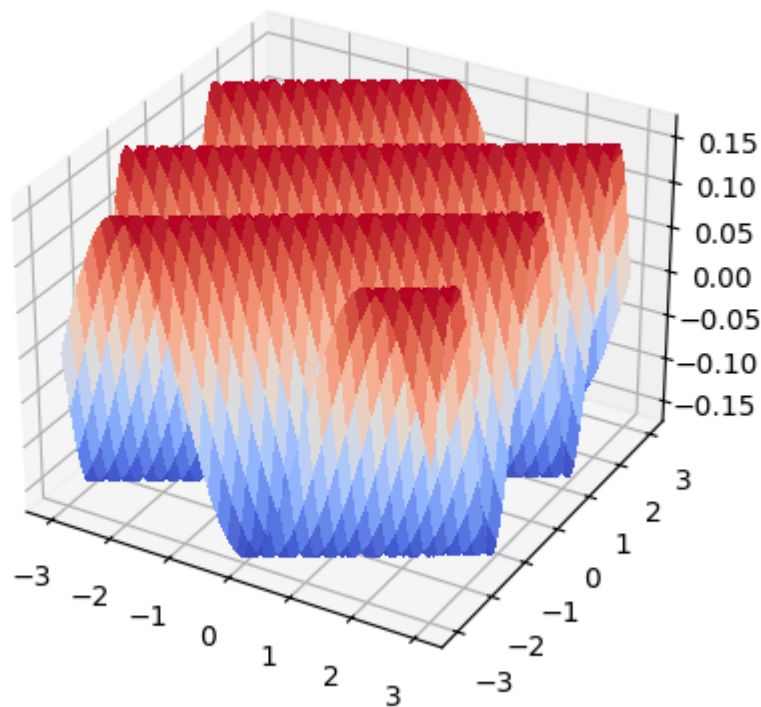
$$\cos(-k_x x + k_y y + k_z z) + j \sin(-k_x x + k_y y + k_z z)$$

and $\cos(-k_x x + k_y y + k_z z) = \cos(k_x x + k_y y + k_z z)$ only if $k_y y + k_z z$ is a multiple of 2π , which is generally not true. This can also be seen in the figures below. The plane waves are periodic only in the direction of the wave vector.

i=1, x_i=-1.5, y_j=2.5 real



i=1, x_i=-1.5, y_j=2.5 imag



Interpolation

As described by eq 23 Interpolating a scalar quantity h at a single point requires a sum over all grid points which may be costly (speaking of working interactively). If h needs to be interpolated on a large number of points, ,

$$\mathbf{r} = \begin{bmatrix} x_0 & y_0 & z_0 \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_{-1} & y_{-1} & z_{-1} \end{bmatrix}$$

it will be advantageous to move the loop over the points $0..-1$ inside the loop over ijk product $h_{ijk}f_i(x)f_j(y)f_k(z)$.

In case h is not a scalar quantity but a tensor it is advantageous to move the loop over the components between the loop over the interpolation points and the loop over ijk :

```
for all grid points ijk:
    for all components h of H
        for all interpolation points 0..p
            accumulate h_ijk * f_i(x_p) * f_j(y_p) * f_k(z_p) over ijk
```

The inner loop can be evaluated as a function over a numpy array:

```
for all grid points ijk:
    for all components h of H
        h(r) += h_ijk * f_i(r[:,0]) * f_j(r[:,1]) * f_k(r[:,2])
```

Furthermore, in case e.g. the x axis is reduced, according to eq 15 one must replace $f_i(x)$ by $(f_i(x) \pm f_{-i}(x))$, where the sign is $+$ if h is symmetric w.r.t. x and $-$ if h is skew-symmetric w.r.t. x .